

# GreenVision

Plant Disease Image Classifier

39 Classes

EfficientNet-B0

98.54% Accuracy

FastAPI · Next.js



# Problem + Stakeholder

## Farmers

Plant diseases destroy up to 40% of global crops. Farmers lack fast, accurate diagnosis — leading to delayed treatment, crop loss, and over-use of pesticides.

# Architecture

aml-greenvision/

```
greenvision/  
  data.py    ← pipeline  
  model.py   ← EfficientNet-B0  
  train.py   ← 2-phase trainer  
app/main.py ← FastAPI /predict  
dashboard/  ← Next.js 15 UI  
artifacts/  ← splits + classes  
models/     ← .pt checkpoints  
tests/      ← pytest fixtures  
mlruns/     ← MLflow experiments
```

**PlantVillage**  
54,305 images

*data.py — stratified 80/10/10*



**EfficientNet-B0**  
Backbone + Head

*model.py — torchvision pretrained*



**2-Phase Training**  
MLflow Tracking

*train.py — freeze → fine-tune*



**FastAPI · Next.js**  
POST /predict

*app/main.py + dashboard/*

# Training Results

98.54%

Top-1 Test Accuracy

5,545 held-out images

99.98%

Top-5 Accuracy

near-perfect retrieval

98.05%

Macro F1

across 39 classes

| Approach                    | Val Accuracy         | Notes                        |
|-----------------------------|----------------------|------------------------------|
| Naive baseline (random)     | 2.6%                 | 1/39 chance                  |
| Phase 1 – frozen backbone   | ~97%                 | Classification head only     |
| Phase 2 – fine-tuned (ours) | 98.34% → 98.54% test | Unfroze all layers, lower LR |

# Two-Phase Training

## PHASE 1 — Head Only

### Goal

Train only the classification head while keeping the pretrained EfficientNet-B0 backbone frozen

### Setup

- Backbone layers: `requires_grad = False`
- New classifier head: 39 output classes
- Higher learning rate for fast convergence

### Result

~97% validation accuracy



## PHASE 2 — Fine-Tune

### Goal

Unfreeze all layers and fine-tune the entire network end-to-end with a lower learning rate

### Setup

- All layers: `requires_grad = True`
- Lower LR (1e-3 to 1e-4) to preserve learned features
- MLflow tracks params + per-epoch metrics

### Result

98.34% val acc → 98.54% test acc  
Checkpoint: `phase2_best.pt` (epoch 2)

# Reflection

## If I Had More Time

### 1. figure out the data bias issue

Keep working through and see if I can produce anything else to “mess up” the data in a meaningful way

### 2. Include suggested fixes

Any prediction with high confidence I would include a suggested fix to the issue

### 3. More plant species and diseases

See if I could find more data on more plants and diseases